TRABALHO DE DISPOSITIVOS MÓVEIS

PRATELEIRA VIVA

Aplicativo para controle de validade de produtos

Lucas Cabrio - 04723-133
Paulo Esteves - 04723-090

Relatório do projeto desenvolvido em React Native com Expo,
backend Node.js e banco de dados SQLite.

Outubro de 2026

1. Problema e proposta

O projeto trata do controle de validade em um pequeno mercado. Nesse tipo de negócio, o mesmo produto pode chegar em entregas diferentes e ter datas de vencimento distintas. Conferir apenas o nome do produto não é suficiente: é necessário acompanhar cada lote.

Foi usado como exemplo o Mercado do Bairro, uma empresa fictícia. O aplicativo é voltado ao responsável pelo estoque, que cadastra os lotes e consulta quais precisam ser vendidos primeiro. A intenção é diminuir os esquecimentos e o descarte de produtos. Ainda não houve uso em uma empresa real.

1.1 Funcionamento

Após entrar na conta, o usuário vê um resumo dos lotes, a quantidade de unidades e os produtos próximos do vencimento. Na consulta, pode buscar pelo nome ou filtrar os lotes que vencem em até sete dias. O cadastro pede nome, quantidade, preço unitário, validade e categoria.

Cada registro representa um lote. Por exemplo, duas caixas de leite com validades diferentes devem ser cadastradas separadamente. Os dados podem ser editados e a exclusão pede confirmação.

1.2 Regras escolhidas

Prazo de validade	Orientação no aplicativo
Já venceu	Retirar da venda; sem desconto
Hoje ou nos próximos 2 dias	Sugerir 30% de desconto
Entre 3 e 7 dias	Sugerir 15% de desconto
Mais de 7 dias	Acompanhar, sem desconto

Os percentuais foram definidos para a demonstração do projeto. São sugestões e não alteram automaticamente o preço em um sistema de vendas.

O indicador **valor em risco** soma quantidade × preço dos lotes que vencem entre hoje e sete dias. Um lote com 10 unidades de R$ 5,00 contribui com R$ 50,00 para o indicador. Esse valor não representa prejuízo já ocorrido.

2. Arquitetura e organização do código

O projeto tem um aplicativo e um backend separados. O aplicativo usa React Native com Expo e se comunica com o backend por requisições HTTP em JSON. O backend funciona como BFF: atende às telas e calcula as informações que elas precisam. Apenas ele acessa o banco e a API externa.

Aplicativo Expo → BFF Node.js → SQLite
O BFF também consulta o Open Food Facts pela internet.

O SQLite foi escolhido porque salva os dados em um arquivo e não exige instalar um servidor de banco separado. O arquivo fica em **server/data.db**. Reiniciar o backend não apaga os cadastros.

2.1 Separação dos arquivos

mobile/src/app: rotas e verificação de acesso.
mobile/src/screens: telas de login, resumo, consulta e formulário.
mobile/src/components: campos, botões, mensagens e cartões de lote.
mobile/src/contexts: sessão e dados usados pelas telas.
mobile/src/services/api.ts: requisições para o backend.

O cartão **BatchCard** aparece no resumo e na consulta. O **BatchFormScreen** serve para cadastrar e editar. Assim, uma alteração no formulário não precisa ser repetida em duas telas.

server/src/app.js: rotas da API.
auth.js: senhas e sessões.
rules.js: validação e regras de validade.
database.js: conexão e criação das tabelas.
seed.js: conta e lotes de demonstração.

2.2 Banco e autenticação

A tabela **users** guarda nome, e-mail e senha protegida por hash. **sessions** relaciona o token ao usuário e ao prazo de expiração. **batches** guarda os lotes e o identificador do proprietário.

O login gera uma sessão válida por 24 horas. As rotas verificam o token e filtram os registros pelo usuário. A senha usa scrypt com salt; o banco guarda somente o hash do token de sessão. No celular, o token fica no SecureStore. Na versão web, fica no sessionStorage.

3. Integração externa e validações

O Open Food Facts é usado para buscar o nome de um alimento pelo código de barras. O código é digitado no formulário e enviado ao BFF, que consulta **/api/v2/product/{codigo}.json**. Não foi implementada leitura pela câmera.

Quando encontra o produto, o app preenche o nome. Quantidade, preço e validade continuam sendo informados pelo usuário, pois são dados específicos daquele lote. Na verificação realizada, o código 3017620422003 retornou Nutella.

Se o catálogo não responder ou não tiver o produto, aparece uma mensagem e o cadastro manual continua disponível. O nome retornado deve ser conferido, pois o catálogo é colaborativo.

3.1 Rotas do BFF

Rota	Uso
POST /auth/register	Cadastro de conta
POST /auth/login	Login
GET /auth/me	Consulta da sessão
POST /auth/logout	Encerramento da sessão
GET /dashboard	Indicadores
GET e POST /batches	Consulta e cadastro de lotes
PUT e DELETE /batches/:id	Edição e exclusão
GET /products/lookup	Consulta ao Open Food Facts

3.2 Tratamento de erros

O formulário verifica campos obrigatórios, quantidade, preço e formato da validade. O backend repete a validação e também rejeita datas inexistentes. Por exemplo, 30/02/2026 não é aceito, mesmo tendo o formato esperado.

A edição e a exclusão conferem se o lote pertence ao usuário autenticado. O acesso ao banco usa consultas com parâmetros. Mensagens de erro são mostradas na tela quando a sessão expira, a rede falha ou os dados estão incorretos.

4. Telas do aplicativo

As figuras foram capturadas na versão web do aplicativo Expo, em uma janela de 430 × 932 pixels. Os registros exibidos são dados de demonstração.



Figura 1 - Login. Entrada por e-mail e senha, com opção de criar conta.



Figura 2 - Resumo. Valor dos lotes próximos da validade, quantidade e lista por vencimento.

4. Telas do aplicativo (continuação)



Figura 3 - Consulta. Busca por produto e filtros de validade.

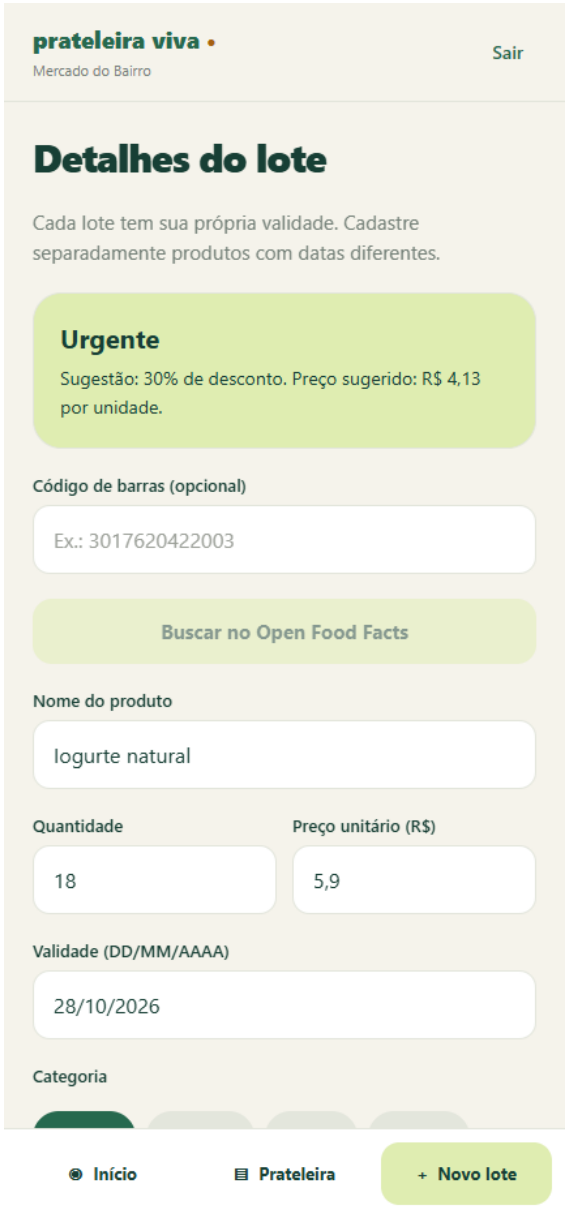


Figura 4 - Edição. Dados do lote e sugestão de desconto calculada pelo backend.

O lote de iogurte mostrado na edição está perto do vencimento. Por isso, recebe a sugestão de 30%. O produto vencido na consulta é sinalizado para retirada da venda, sem sugestão de desconto.

4. Telas do aplicativo (continuação)



Figura 5 - Cadastro. Preenchimento dos dados de um novo lote.

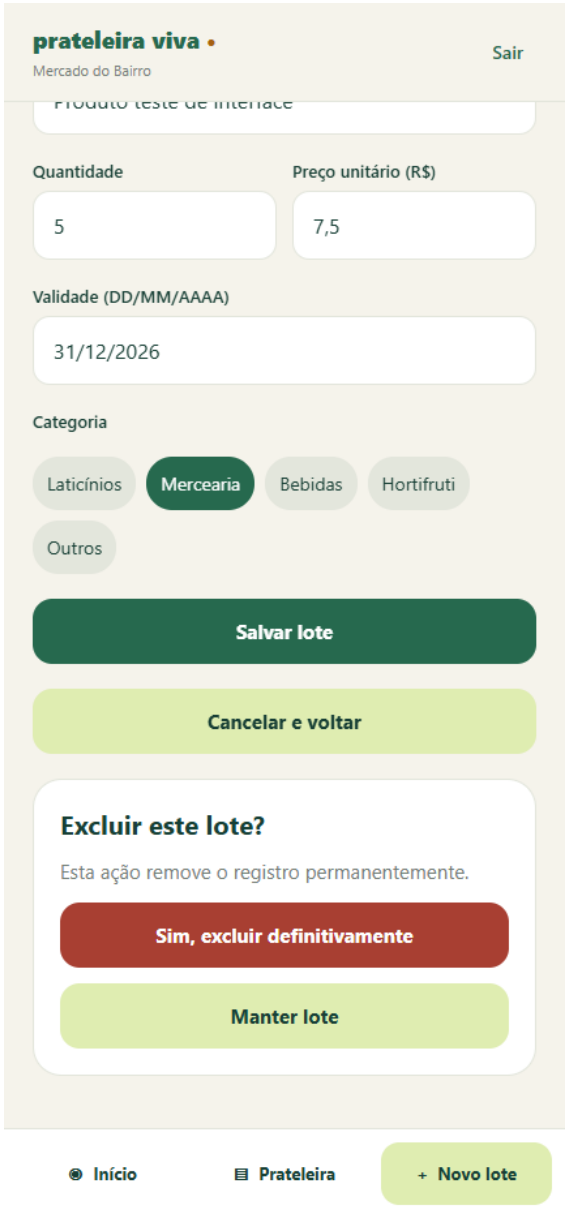


Figura 6 - Exclusão. Confirmação antes de remover o registro.

O formulário de cadastro e edição é o mesmo componente. Na edição, os campos começam preenchidos. Após salvar ou excluir, o app atualiza a lista e mostra uma mensagem com o resultado.

5. Execução e testes

É necessário ter Node.js 24 LTS e npm. Na pasta do projeto, executar:

```
npm --prefix mobile ci
npm run seed
npm run backend
```

Em outro terminal: `npm run web`. A conta de demonstração é **demo@prateleiraviva.com**, com senha **Demo@12345**. O seed não sobrescreve dados existentes.

No celular, EXPO_PUBLIC_API_URL deve apontar para o IP do computador, na porta 3001. Os dois precisam estar na mesma rede. O README traz a configuração para Expo Go compatível e para development build.

5.1 Verificações realizadas

Os testes do backend cobrem cadastro, login, dados inválidos, CRUD, separação dos registros por usuário, expiração e logout. Também verificam a leitura dos dados depois de fechar e reabrir o banco. A consulta externa foi testada com respostas controladas de sucesso e erro, além de uma chamada real.

O fluxo de interface foi executado no navegador: login, cadastro, consulta, edição, exclusão e logout. TypeScript e ESLint passaram. O Expo Doctor aprovou 20 verificações e foram gerados bundles Android, iOS e web. Ainda falta testar em aparelhos físicos; não foi gerado APK ou IPA.

5.2 Limitações

Cada usuário tem seus próprios lotes; não há compartilhamento por loja. Também não há registro de vendas, notificações ou funcionamento offline. A aplicação usa HTTP local para a apresentação. Um uso real exigiria hospedagem com HTTPS, backup e ajustes de acesso.

A verificação de dependências apontou três alertas de segurança na cadeia Expo CLI/Zod. A correção automática sugeria reduzir a versão do Expo e não foi aplicada. O detalhe está registrado no README.

Referências

Expo: https://docs.expo.dev/versions/v58.0.0/
Expo Router: https://docs.expo.dev/router/installation/
Open Food Facts: https://openfoodfacts.github.io/openfoodfacts-server/api/
SQLite no Node.js: https://nodejs.org/api/sqlite.html